Software Renewals, Contracts & OEM Bundle Tracking

Implementation Solution Guide

Version 1.0.0 · 2026-06-29

Contents

Executive Summary — Software Renewals, Contracts & OEM Bundle Tracking 4

What you get . . . 4

What this delivers . . . 4

The standard deployment model . . . 4

Outcome . . . 4

1. Solution Overview — Software Renewals, Contracts & OEM Bundle Tracking 5

The problem this solves . . . 5

The renewal lifecycle . . . 5

The contract and entitlement repository . . . 5

OEM bundle mapping . . . 5

Lead-time alerting . . . 6

The procurement coordination workflow . . . 6

What this delivers (the named artifacts) . . . 6

Scope and honest limits . . . 6

The standard deployment model . . . 7

2. Requirements Guide — Software Renewals, Contracts & OEM Bundle Tracking 7

2.1 Contract and entitlement source documents . . . 7

2.2 A controlled repository location . . . 7

2.3 Renewal dates and contract terms captured . . . 7

2.4 OEM and bundle relationships known . . . 7

2.5 An alerting mechanism . . . 8

2.6 Named renewal owners and a procurement liaison . . . 8

2.7 Subscription data access (where available) . . . 8

Checklist . . . 8

3. Implementation Guide — Software Renewals, Contracts & OEM Bundle Tracking 9

Step 1 — Inventory contracts and renewal dates (Assess) . . . 9

Step 2 — Build the contract and entitlement repository (Configure) . . . 9

Step 3 — Build the renewal calendar (Configure) . . . 10

Step 4 — Map OEM and suite bundles (Configure) . . . 10

Step 5 — Arm lead-time alerts (Configure) . . . 10

Step 6 — Define the procurement coordination workflow (Configure) . . . 10

Step 7 — Pilot (Pilot) . . . 11

Step 8 — Validate the pilot (Validate) — pilot gate . . . 11

Step 9 — Expand to the full portfolio (Expand) . . . 11

5. Testing Guide — Software Renewals, Contracts & OEM Bundle Tracking 11

Test 1 — Repository complete . . . 11
Test 2 — Calendar reflects the repository . . . 12
Test 3 — OEM bundles collapse to one decision . . . 12
Test 4 — Lead-time alerts fire at the right tier . . . 12
Test 5 — A renewal routes through the decision workflow . . . 12
Test 6 — Closing updates the record (safe) . . . 12
Test 7 — Owners route correctly . . . 12
Pass/fail gate . . . 12
6. Troubleshooting Guide — Software Renewals, Contracts & OEM Bundle Tracking 13
A. Renewals not appearing on the calendar . . . 13
B. Alerts firing late, early, or not at all . . . 13
C. OEM bundle problems . . . 13
D. Validation reports incomplete records . . . 14
E. Discovery returns no subscription data . . . 14
F. Remediation made an unexpected change . . . 14
G. Diagnostics quick reference . . . 14
H. When to escalate . . . 14
7. Best Practices — Software Renewals, Contracts & OEM Bundle Tracking 14
8. Deliverables Summary — Software Renewals, Contracts & OEM Bundle Tracking 15
Documentation . . . 15
PowerShell scripts . . . 16
Named artifacts produced . . . 16
What the buyer receives . . . 16
Outcome . . . 16
Appendix A — PowerShell Scripts 18

Executive Summary — Software Renewals, Contracts & OEM Bundle Tracking

Renewals are where software money leaks and risk hides. They arrive without warning, auto-renew at the wrong quantity or worse terms, or quietly lapse and break something in production. The contract terms, OEM bundle entitlements, and renewal dates that should drive these decisions live scattered across inboxes, PDFs, and people's memories. The result is a string of surprises instead of a series of planned decisions.

This is a complete, do-it-today package for building a repeatable software renewals, contracts, and OEM bundle tracking process, written for a SAM analyst with no prior renewal-management framework in place. It turns the renewal lifecycle into a tracked, alerted, owned workflow.

What you get

- A plain-language Solution Overview of the renewal lifecycle, the contract and entitlement

repository model, OEM bundle mapping, lead-time alerting, and the procurement coordination workflow.

- A Requirements Guide with verify / implement-if-missing for every prerequisite.
- An Implementation Guide of precise, ordered steps: inventory contracts, build the repository and

calendar, map OEM bundles, set lead-time alerts, define the renewal decision workflow, then pilot, validate, and expand.

- Five PowerShell scripts for contract and renewal discovery, renewal-data completeness validation,

a renewal calendar and cost report, safe remediation of missing renewal records, and a renewal-owner group builder.

- Testing and Troubleshooting guides with explicit pass/fail gates, plus field-proven Best Practices.

What this delivers

A named, durable set of artifacts: a `Renewal Calendar – All Products – Live` driven by a `Contract & Entitlement Repository – All Products – Master`, an `OEM Bundle Map – Vendor Bundles – Master`, a set of `Lead-Time Alert Rules – Renewals – Tiered` (T-120 / T-90 / T-60 / T-30), and a `Renewal Decision Workflow – Per Contract – Standard` that routes every upcoming renewal to a named owner and procurement liaison for a planned keep / change / drop decision.

The standard deployment model

Assess → Configure → Pilot → Validate → Expand, with a pilot gate before any expansion.

Outcome

Every software renewal becomes a planned decision — no surprise auto-renewals, no lapses that break production, no terms trapped in inboxes — proven on a pilot product family before you scale it across the portfolio.

1. Solution Overview — Software Renewals, Contracts & OEM Bundle Tracking

This is the only section that explains the *why*. Every later section is "do this" steps.

The problem this solves

Software renewals fail in four predictable ways. They arrive with no warning, so there is no time to negotiate or budget. They auto-renew at the wrong quantity or on worse terms because nobody reviewed them. They lapse and break a production dependency because the date slipped past unseen. And the facts needed to decide — contract terms, true-up clauses, OEM bundle entitlements, the renewal date itself — sit in inboxes and PDFs no one can find. The fix is not heroics near each deadline; it is a repeatable process that makes every renewal a planned decision, far enough ahead to act on.

The renewal lifecycle

A renewal is not an event, it is a lifecycle with stages. The repeatable way tracks each contract through:

1. Inventory — the contract, its products, quantity, cost, term, and renewal date are captured in the repository.
1. Watch — the renewal date is on the calendar and lead-time alerts are armed.
2. Alert — at each lead-time threshold the named owner is notified that a decision is approaching.
3. Decide — the owner and procurement liaison run the decision workflow: keep, change (quantity or terms), consolidate, or drop.
1. Act — procurement executes the decision (renew, renegotiate, or let lapse intentionally).
2. Close — the repository is updated with the new term, cost, and next renewal date, and the cycle restarts.

The contract and entitlement repository

The single source of truth is the `Contract & Entitlement Repository – All Products – Master`. It is one controlled record per contract holding the fields the lifecycle needs: vendor/publisher, product, quantity, unit and total cost, contract start and end, renewal/notice dates, auto-renew flag, notice period, true-up terms, the named renewal owner, the procurement liaison, and a link to the contract document. A controlled spreadsheet or a SAM/CMDB table both work; the schema, not the tool, is what matters. The repository feeds everything else — the calendar, the alerts, and the cost report all read from it.

OEM bundle mapping

OEM and suite bundles hide entitlements: a single purchase (a device OEM bundle, a publisher suite, a volume agreement) often grants several products with one shared renewal or one shared expiry. If they are tracked as separate line items, you double-count cost, miss that dropping one product strands the others, or renew a component that the bundle already covers. The `OEM Bundle Map – Vendor`

`Bundles – Master` records each bundle, the parent SKU, the child entitlements it contains, and the single renewal anchor — so the calendar shows one decision, not five disconnected ones.

Lead-time alerting

A renewal date with no warning is useless. The `Lead-Time Alert Rules – Renewals – Tiered` fire at tiered thresholds before each renewal — T-120, T-90, T-60, and T-30 days — so high-value or contractual-notice renewals surface early enough to negotiate, and routine ones still get a final reminder. Each alert names the renewal owner and the procurement liaison and points at the contract record, so the recipient can act without hunting for context.

The procurement coordination workflow

Alerts without a decision path just create noise. The `Renewal Decision Workflow – Per Contract – Standard` defines what happens when an alert fires: the renewal owner reviews usage and need against the repository, classifies the renewal (keep as-is / change quantity or terms / consolidate into a bundle / drop), and hands a clear recommendation to the procurement liaison, who executes and records the outcome. Every renewal closes with a documented decision and a refreshed next renewal date.

What this delivers (the named artifacts)

- `Contract & Entitlement Repository – All Products – Master` — the single source of truth.
- `Renewal Calendar – All Products – Live` — a date-ordered view of every upcoming renewal.
- `OEM Bundle Map – Vendor Bundles – Master` — parent SKUs to child entitlements with one anchor.
- `Lead-Time Alert Rules – Renewals – Tiered` — T-120 / T-90 / T-60 / T-30 notifications.
- `Renewal Decision Workflow – Per Contract – Standard` — the keep / change / consolidate / drop path.
- `Renewal Owners – Pilot Scope – Group` and `Renewal Owners – All Products – Group` — who owns each

renewal, used for alert routing.

Scope and honest limits

- This solution governs the renewal *process* and its records. It reads subscription/renewal data

from Microsoft Graph where the product exposes it, and otherwise relies on a CSV repository you maintain — it cannot invent a renewal date a vendor never gave you.

- It does not negotiate contracts or move money; it makes sure the right person decides in time.
- Microsoft Graph surfaces Microsoft 365 subscription/SKU data, not every third-party publisher's

contract terms — those are captured in the repository by hand or by vendor export.

The standard deployment model

> *Inventory contracts → build the repository and calendar → map OEM bundles → arm lead-time alerts → define the decision workflow → validate on a pilot product family → expand to the full portfolio. Deployment model: Assess → Configure → Pilot → Validate → Expand.*

2. Requirements Guide — Software Renewals, Contracts & OEM Bundle Tracking

Verify every requirement before configuring. Each entry: Description · Why · Verify · Implement-if-missing. Category-wide prerequisites are in `global-requirements-asset-mgt.md` (G1–G7); the items below are specific to the renewal, contract, and OEM-bundle process.

2.1 Contract and entitlement source documents

Description: Access to the actual contracts, order forms, true-up statements, and OEM/bundle agreements for the products in scope. Why: The repository can only be as accurate as the documents it is built from; renewal dates, notice periods, and bundle entitlements come from these. Verify: You can open a representative contract and read its term, renewal date, and notice period. Implement-if-missing: Collect contracts from procurement, finance, and vendor portals; for missing documents, request a renewal/entitlement statement from each vendor or reseller.

2.2 A controlled repository location

Description: A maintained, access-controlled store for the `Contract & Entitlement Repository – All Products – Master` — a SAM/ITAM tool, a CMDB table, or a controlled spreadsheet/CSV. Why: The repository is the single source of truth the calendar, alerts, and cost report all read from. Verify: You can locate and edit a controlled record with the repository schema (see Step 2 of the Implementation Guide); access is limited to the SAM team. Implement-if-missing: Stand up the repository using the documented schema; restrict edit rights to the SAM analyst and approver; keep a version history.

2.3 Renewal dates and contract terms captured

Description: For each in-scope contract: renewal date, notice/cancellation period, auto-renew flag, quantity, and cost. Why: These five fields drive the calendar and the lead-time alerts; without them a renewal cannot be watched or decided. Verify: Run `SWRENEW-Validation.ps1` against the repository CSV — it flags records missing any of these fields. Implement-if-missing: Fill the gaps from the source documents; for truly unknown dates, request them from the vendor and use `SWRENEW-Remediation.ps1` to insert placeholder records pending confirmation.

2.4 OEM and bundle relationships known

Description: Knowledge of which purchases are OEM/suite/volume bundles and which child entitlements each one contains. Why: Bundles share a single renewal anchor; tracking children separately double-counts cost and hides dependencies. Verify: You can identify, for a representative bundle, its parent SKU and the products it entitles. Implement-if-missing: Build the `OEM Bundle Map – Vendor`

`Bundles – Master` from order forms and vendor SKU documentation; link each child entitlement to its parent and shared renewal anchor.

2.5 An alerting mechanism

Description: A way to deliver the `Lead-Time Alert Rules – Renewals – Tiered` notifications — email, a ticketing queue, a calendar with reminders, or a scheduled report. Why: A renewal date with no notification is invisible; the alert is what triggers the decision workflow. Verify: You can send a test reminder to a renewal owner and confirm receipt. Implement-if-missing: Choose the channel (a shared mailbox, a ServiceNow queue, or scheduled `SWRENEW-Report.ps1` output) and document who receives each tier.

2.6 Named renewal owners and a procurement liaison

Description: A named owner per contract (or product family) and a procurement contact who executes decisions. Why: The decision workflow routes each renewal to a person; unrouted renewals stall. Verify: For a representative contract you can name the renewal owner and the procurement liaison. Implement-if-missing: Assign owners in the repository; build `Renewal Owners – Pilot Scope – Group` and `Renewal Owners – All Products – Group` with `SWRENEW-Set-ScopeTargeting.ps1` for routing.

2.7 Subscription data access (where available)

Description: Microsoft Graph read access to subscribed-SKU / subscription data for products that expose it (e.g. Microsoft 365). Why: Where a vendor exposes renewal/subscription data programmatically, the discovery script can seed the repository instead of manual entry. Verify: `SWRENEW-Discovery.ps1` connects to Graph and returns subscribed SKUs. Implement-if-missing: Grant the read scope (`Organization.Read.All` / `Directory.Read.All`) and install the Microsoft Graph PowerShell SDK; for vendors with no API, rely on the CSV repository.

Checklist

- Contract / order / true-up / bundle documents collected
- Controlled repository location stood up (schema in Implementation Step 2)
- Renewal date, notice period, auto-renew, quantity, cost captured — `SWRENEW-Validation.ps1`
- OEM/bundle parent-child relationships known
- Alerting channel chosen and tested
- Named renewal owners + procurement liaison assigned
- Microsoft Graph subscription read access (where the product exposes it) — `SWRENEW-Discovery.ps1`
- Pilot product family selected (G7)

3. Implementation Guide — Software Renewals, Contracts & OEM Bundle Tracking

Follow these steps in order. Do the work in your repository tool (a controlled spreadsheet/CSV, a SAM/ITAM tool, or a CMDB) and run the scripts from an elevated PowerShell prompt. Names in `backticks` are the named artifacts — create them with these exact names and reuse them verbatim.

> *Model: build the repository and calendar once, then run the renewal lifecycle the same way every cycle. Prove it on a pilot product family before expanding. Deployment model: Assess → Configure → Pilot → Validate → Expand.*

Step 1 — Inventory contracts and renewal dates (Assess)

1. Collect every in-scope contract, order form, true-up statement, and OEM/bundle agreement.

2. For products that expose data via Microsoft Graph, run `SWRENEW-Discovery.ps1 -CsvPath .\renewals-seed.csv` to seed subscribed SKUs into a starter CSV.

1. For all other products, add a row per contract from the source documents.

2. Result of this step: a draft list of contracts with vendor, product, quantity, cost, and the renewal date where known.

Step 2 — Build the contract and entitlement repository (Configure)

1. Create the `Contract & Entitlement Repository – All Products – Master` with this schema (one row per contract). These exact column names are read by the scripts:

- `ContractId` — a unique key per contract.
- `Vendor` — publisher / reseller.
- `Product` — product or SKU name.
- `Quantity` — licensed quantity.
- `UnitCost` and `AnnualCost` — per-unit and total annual cost.
- `Currency` — e.g. USD.
- `TermStart` and `RenewalDate` — ISO dates (`YYYY-MM-DD`).
- `NoticePeriodDays` — days of notice required to cancel/change.
- `AutoRenew` — `Yes` or `No`.
- `BundleId` — blank, or the parent bundle key from Step 4.
- `Owner` — the named renewal owner.
- `ProcurementLiaison` — the procurement contact.
- `DocumentLink` — path/URL to the contract document.

1. Import the draft list from Step 1. Restrict edit rights to the SAM analyst and approver.

2. Run `SWRENEW-Validation.ps1 -CsvPath .\repository.csv` to confirm every record has the fields the

lifecycle needs.

Step 3 — Build the renewal calendar (Configure)

1. Create the `Renewal Calendar – All Products – Live` as a date-ordered view of the repository,

sorted by `RenewalDate` ascending, showing `Vendor`, `Product`, `Quantity`, `AnnualCost`, `RenewalDate`, `NoticePeriodDays`, `AutoRenew`, and `Owner`.

1. Generate it from the repository with `SWRENEW-Report.ps1 -CsvPath .\repository.csv -OutDir .\reports` (produces the calendar and a cost rollup). Refresh it on a schedule (Step 6).

Step 4 — Map OEM and suite bundles (Configure)

1. Create the `OEM Bundle Map – Vendor Bundles – Master` with: `BundleId`, `ParentSku`, `ChildProduct`, `RenewalAnchorDate`, and `Notes` (one row per child entitlement).

1. For each bundle, list its child entitlements under one `BundleId` and one `RenewalAnchorDate`.

2. In the repository, set `BundleId` on every contract row that belongs to a bundle so the calendar

collapses the bundle into a single renewal decision.

Step 5 — Arm lead-time alerts (Configure)

1. Define the `Lead-Time Alert Rules – Renewals – Tiered` as four thresholds before `RenewalDate`:

T-120, T-90, T-60, and T-30 days.

1. Use T-120/T-90 for high-value contracts and any with a long `NoticePeriodDays`; T-60/T-30 catch routine renewals. The notice period must always fall inside an active alert window — if `NoticePeriodDays` exceeds 120, raise the earliest tier for that contract.

1. Deliver alerts through the channel chosen in Requirement 2.5. Schedule `SWRENEW-Report.ps1` to emit

the due-soon list per tier; each alert names the `Owner` and `ProcurementLiaison`.

Step 6 — Define the procurement coordination workflow (Configure)

1. Document the `Renewal Decision Workflow – Per Contract – Standard`:

• On each alert, the `Owner` reviews usage and need against the repository.

• The `Owner` classifies the renewal: keep as-is / change quantity or terms / consolidate into a bundle / drop.

• The `Owner` hands the recommendation to the `ProcurementLiaison`, who executes and records the outcome and the new `RenewalDate`.

1. Build the routing groups with `SWRENEW-Set-ScopeTargeting.ps1`: `Renewal Owners – Pilot Scope – Group for the pilot, and Renewal Owners – All Products – Group` for the full portfolio.

Step 7 — Pilot (Pilot)

1. Pick one product family (G7) as the pilot scope and confine the calendar and alerts to `Renewal Owners – Pilot Scope – Group`.

1. Run one full cycle: let an alert fire, run the decision workflow, execute through the liaison, and close the record with `SWRENEW-Remediation.ps1 -WhatIf` first, then without `-WhatIf` to write the updated renewal record.

1. Complete the Testing Guide for the pilot.

Step 8 — Validate the pilot (Validate) — pilot gate

1. Confirm the pilot passed the Testing Guide pass/fail gate (every pilot renewal had an alert, a documented decision, and a refreshed `RenewalDate`).

1. Do not expand until this gate is green.

Step 9 — Expand to the full portfolio (Expand)

1. Switch the calendar and alerts to `Renewal Owners – All Products – Group`.

2. Run the same cycle for every contract; re-validate each cycle with `SWRENEW-Validation.ps1` and refresh the calendar/cost report with `SWRENEW-Report.ps1`.

1. Review the cost rollup quarterly to spot consolidation and drop opportunities.

Order of operations

`Inventory → Repository → Calendar → OEM bundle map → Lead-time alerts → Decision workflow → Pilot → Validate (gate) → Expand`

5. Testing Guide — Software Renewals, Contracts & OEM Bundle Tracking

Run after the pilot is configured (Implementation Step 7) and again each renewal cycle. Use the pilot scope (`Renewal Owners – Pilot Scope – Group`) until the gate passes.

Test 1 — Repository complete

Do: Run `SWRENEW-Validation.ps1 -CsvPath .\repository.csv`. Expected: Every record reports complete — no missing `RenewalDate`, `NoticePeriodDays`, `AutoRenew`, `Quantity`, or `AnnualCost`. Result line reads REPOSITORY COMPLETE.

Test 2 — Calendar reflects the repository

Do: Run `SWRENEW-Report.ps1 -CsvPath .\repository.csv -OutDir .\reports` and open the `Renewal Calendar – All Products – Live` output. Expected: Every in-scope contract appears, sorted by `RenewalDate`, with owner and cost shown.

Test 3 — OEM bundles collapse to one decision

Do: Inspect a bundled contract in the calendar. Expected: The bundle's child entitlements share one `BundleId` and one renewal anchor — the calendar shows a single renewal decision, not one per child.

Test 4 — Lead-time alerts fire at the right tier

Do: With a contract whose `RenewalDate` is within 120 / 90 / 60 / 30 days, run `SWRENEW-Report.ps1`. Expected: The contract appears in the correct due-soon tier, and its notice period falls inside the alert window. The alert names the `Owner` and `ProcurementLiaison`.

Test 5 — A renewal routes through the decision workflow

Do: Take one alerted pilot renewal through the `Renewal Decision Workflow – Per Contract – Standard`. Expected: The owner records a keep / change / consolidate / drop decision and the liaison records the outcome — a documented decision exists before the renewal date.

Test 6 — Closing updates the record (safe)

Do: Run `SWRENEW-Remediation.ps1 -CsvPath .\repository.csv -WhatIf`, review the proposed changes, then run without `-WhatIf`. Expected: With `-WhatIf`, changes are previewed only. Without it, the closed renewal's `RenewalDate` advances to the next term and missing records are filled. A backup of the CSV is written first.

Test 7 — Owners route correctly

Do: Run `SWRENEW-Set-ScopeTargeting.ps1` and review the `Renewal Owners – Pilot Scope – Group`. Expected: Each contract maps to a named owner; alerts reach the right person.

Pass/fail gate

Gate	Requirement
Pilot pass	Tests 1–5 pass; every pilot renewal had an alert, a documented decision, and a refreshed `RenewalDate`
Expand pass	Tests 1, 2, 4, 7 pass for the full portfolio before the next quarterly review

6. Troubleshooting Guide — Software Renewals, Contracts & OEM Bundle Tracking

Use `SWRENEW-Validation.ps1` for repository completeness, `SWRENEW-Report.ps1` for the calendar and due-soon tiers, `SWRENEW-Discovery.ps1` to re-seed from Graph, and `SWRENEW-Remediation.ps1` (default `-WhatIf`) for safe fixes.

A. Renewals not appearing on the calendar

Symptom	Cause	Remediation
A known contract is missing from the calendar	No record in the repository, or a blank `RenewalDate`	Add the contract; run `SWRENEW-Validation.ps1` to find blank dates; fill from the source document
Calendar is empty or stale	Report run against the wrong CSV, or not refreshed	Re-run `SWRENEW-Report.ps1 -CsvPath .\repository.csv -OutDir .\reports`
Contract shows but in the wrong order	`RenewalDate` not in `YYYY-MM-DD` ISO format	Correct the date format; re-run the report

B. Alerts firing late, early, or not at all

• Alert never fires: the contract has no `RenewalDate`, or the alerting schedule is not running. Fix the date and confirm the scheduled `SWRENEW-Report.ps1` job is active.

• Alert fires too late to give notice: `NoticePeriodDays` is longer than the earliest active tier. Raise the earliest tier for that contract so the notice window sits inside an alert.

• Alert reaches the wrong person: `Owner` is wrong or the routing group is stale. Update the repository and re-run `SWRENEW-Set-ScopeTargeting.ps1`.

C. OEM bundle problems

Symptom	Cause	Remediation
Cost looks double-counted	Child entitlements tracked as separate contracts	Set a shared `BundleId` on each child row; map them in the `OEM Bundle Map – Vendor Bundles – Master`
Dropping one product broke another	Bundle dependency not mapped	Record the parent SKU and all child entitlements; treat the bundle as one decision
Bundle renews twice	Two anchor dates for one bundle	Set a single `RenewalAnchorDate` for the `BundleId`

D. Validation reports incomplete records

- `SWRENEW-Validation.ps1` flags missing `RenewalDate`, `NoticePeriodDays`, `AutoRenew`, `Quantity`,

or `AnnualCost`. Fill from source documents; for truly unknown dates, request from the vendor and use `SWRENEW-Remediation.ps1` to insert a placeholder record pending confirmation.

E. Discovery returns no subscription data

- `SWRENEW-Discovery.ps1` returns nothing from Graph: the read scope is missing or the product does

not expose subscription data. Grant `Organization.Read.All` / `Directory.Read.All`; for vendors with no API, populate the repository from contracts manually.

F. Remediation made an unexpected change

- Always preview first with `-WhatIf`. If a write was wrong, restore from the timestamped backup CSV

the script writes before any change (same folder as the repository).

G. Diagnostics quick reference

```
Import-Csv .\repository.csv | Where-Object { -not $_.RenewalDate }        # rows with no renewal date
Import-Csv .\repository.csv | Sort-Object { [datetime]$_.RenewalDate } |  # next renewals
  Select-Object Vendor,Product,RenewalDate,NoticePeriodDays -First 10
Import-Csv .\repository.csv | Group-Object BundleId                       # bundle groupings
```

H. When to escalate

If the repository is complete, alerts are firing on time, owners are correct, and a renewal still slipped, review the alerting channel's delivery (mailbox/queue health) and the decision-workflow hand-off — the gap is in delivery or hand-off, not in the data.

7. Best Practices — Software Renewals, Contracts & OEM Bundle Tracking

- One source of truth. Keep the `Contract & Entitlement Repository – All Products – Master` the

single authoritative record; never let a renewal date live only in an inbox or a calendar invite.

- Lead time beats heroics. Arm the `Lead-Time Alert Rules – Renewals – Tiered` at T-120 / T-90 /

T-60 / T-30 so high-value and long-notice renewals surface early enough to negotiate, not just to rubber-stamp.

- Mind the notice period. The cancellation/notice window must always sit inside an active alert tier;

for any contract whose `NoticePeriodDays` exceeds 120, raise the earliest alert further out.

- Map bundles before you track parts. Record the `OEM Bundle Map – Vendor Bundles – Master` so a

bundle is one renewal decision with one anchor — never track child entitlements as if they renew independently.

- Decide, don't drift. Every alert ends in a documented keep / change / consolidate / drop decision

through the `Renewal Decision Workflow – Per Contract – Standard`; an auto-renew you reviewed is fine, an auto-renew nobody saw is the failure mode.

- Name an owner for every contract. Unrouted renewals stall; assign `Owner` and `ProcurementLiaison`

in the repository and keep the routing groups current.

- Treat a deliberate lapse as a decision. Letting a contract drop is a valid outcome — but only when

the workflow confirmed nothing in production depends on it.

- Pilot before portfolio. Prove one full cycle on a pilot product family and pass the gate before

expanding; the process should be boring before it is broad.

- Reconcile cost every quarter. Use the cost rollup from `SWRENEW-Report.ps1` to find over-licensing,

duplicate coverage inside bundles, and consolidation opportunities.

- Keep the repository honest. Re-run `SWRENEW-Validation.ps1` each cycle and close every renewal by

advancing `RenewalDate`; a stale repository quietly becomes useless.

- Back up before you write. Always preview `SWRENEW-Remediation.ps1` with `-WhatIf` and keep the

automatic CSV backup it creates, so repository edits are reversible.

8. Deliverables Summary — Software Renewals, Contracts & OEM Bundle Tracking

Documentation

Document	Purpose
Executive Summary	Business case and what's in the box
1. Solution Overview	The renewal lifecycle, repository model, OEM bundle mapping, lead-time alerting, procurement workflow, benefits, and limits
2. Requirements Guide	Every prerequisite with Verify / Implement-if-missing
Global Requirements (Asset Mgt)	Category-wide prerequisites, published once

Document	Purpose
3. Implementation Guide	Exact, ordered steps to build the repository, calendar, bundle map, alerts, and workflow
5. Testing Guide	Validation procedures with a pilot pass/fail gate
6. Troubleshooting Guide	Field issues and remediation
7. Best Practices	Field-proven recommendations

PowerShell scripts

Script	What it does
`SWRENEW-Discovery.ps1`	Seeds the repository from Microsoft Graph subscribed SKUs (where exposed) and reports an existing CSV repository's posture. Read-only.
`SWRENEW-Validation.ps1`	Checks every repository record for the fields the renewal lifecycle needs. Read-only.
`SWRENEW-Report.ps1`	Builds the renewal calendar (date-ordered, tiered due-soon) and a cost rollup. Read-only.
`SWRENEW-Remediation.ps1`	Safe (default `-WhatIf`) fill of missing renewal records and advance of closed renewals; backs up the CSV first.
`SWRENEW-Set-ScopeTargeting.ps1`	Builds the pilot and all-products renewal-owner routing groups.

Named artifacts produced

- `Contract & Entitlement Repository – All Products – Master`
- `Renewal Calendar – All Products – Live`
- `OEM Bundle Map – Vendor Bundles – Master`
- `Lead-Time Alert Rules – Renewals – Tiered` (T-120 / T-90 / T-60 / T-30)
- `Renewal Decision Workflow – Per Contract – Standard`
- `Renewal Owners – Pilot Scope – Group` and `Renewal Owners – All Products – Group`

What the buyer receives

- This implementation guide as a Word document and PDF.
- All five PowerShell scripts in a single ZIP.
- A repeatable process following the model: Assess → Configure → Pilot → Validate → Expand.

Outcome

Every software renewal a planned decision — a live renewal calendar with tiered lead-time alerts, a contract and entitlement repository, OEM bundles mapped to one anchor, and a procurement workflow that routes each renewal to a named owner. No surprise auto-renewals, no lapses that break production.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

SWRENEW-Discovery.ps1

Purpose: Reads subscription/SKU data from Microsoft Graph where the product exposes it (e.g. Microsoft 365 subscribed SKUs) to seed a starter renewal repository CSV, and — when given an existing repository CSV — reports its posture (record count, how many are missing renewal dates, and the next renewals due). Read-only: it never edits an existing repository. HOW TO RUN Seed from Graph: powershell -ExecutionPolicy Bypass -File .\SWRENEW-Discovery.ps1 -CsvPath .\renewals-seed.csv Report a CSV: powershell -ExecutionPolicy Bypass -File .\SWRENEW-Discovery.ps1 -ExistingCsv .\repository.csv Graph seeding requires the Microsoft Graph PowerShell SDK and Organization.Read.All. EXPECTED RESULTS A starter CSV of subscribed SKUs (when -CsvPath is given) and/or a posture summary of an existing repository (record count, blank-renewal count, next renewals). .NOTES Software Asset Management — repeatable renewal/contract process. Read-only. #> [CmdletBinding()] param([string]$CsvPath, [string]$ExistingCsv)

How to run: Seed from Graph: powershell -ExecutionPolicy Bypass -File .\SWRENEW-Discovery.ps1 -CsvPath .\renewals-seed.csv Report a CSV: powershell -ExecutionPolicy Bypass -File .\SWRENEW-Discovery.ps1 -ExistingCsv .\repository.csv Graph seeding requires the Microsoft Graph PowerShell SDK and Organization.Read.All. EXPECTED RESULTS A starter CSV of subscribed SKUs (when -CsvPath is given) and/or a posture summary of an existing repository (record count, blank-renewal count, next renewals). .NOTES Software Asset Management — repeatable renewal/contract process. Read-only. #> [CmdletBinding()] param([string]$CsvPath, [string]$ExistingCsv)

SWRENEW-Remediation.ps1

Purpose: Fixes two repository gaps the lifecycle depends on, safely: (1) inserts placeholder values for missing fields (a default NoticePeriodDays and AutoRenew=No) so incomplete records are visible and trackable pending confirmation, and (2) for renewals marked closed this cycle, advances RenewalDate by one term so the next cycle is scheduled. Runs in preview mode by default (-WhatIf); writes only when -WhatIf is removed, and backs up the CSV first. HOW TO RUN Preview (default): powershell -ExecutionPolicy Bypass -File .\SWRENEW-Remediation.ps1 -CsvPath .\repository.csv -WhatIf Apply changes: powershell -ExecutionPolicy Bypass -File .\SWRENEW-Remediation.ps1 -CsvPath .\repository.csv EXPECTED RESULTS With -WhatIf: a list of the changes that WOULD be made, no file written. Without -WhatIf: a timestamped backup is written next to the CSV, then the repository is updated and saved. .NOTES Software Asset Management — repeatable renewal/contract process. Default -WhatIf (safe). #> [CmdletBinding(SupportsShouldProcess, ConfirmImpact='Medium')] param([Parameter(Mandatory)][string]$CsvPath, [int]$DefaultNoticePeriodDays = 30, [int]$TermMonths = 12)

How to run: Preview (default): powershell -ExecutionPolicy Bypass -File .\SWRENEW-Remediation.ps1 -CsvPath .\repository.csv -WhatIf Apply changes: powershell -ExecutionPolicy Bypass -File .\SWRENEW-Remediation.ps1 -CsvPath .\repository.csv EXPECTED RESULTS With -WhatIf: a list of the changes that WOULD be made, no file written. Without -WhatIf: a timestamped backup is written next to the CSV, then the repository is updated and saved. .NOTES Software Asset Management — repeatable renewal/contract process. Default -WhatIf (safe). #> [CmdletBinding(SupportsShouldProcess, ConfirmImpact='Medium')] param([Parameter(Mandatory)][string]$CsvPath, [int]$DefaultNoticePeriodDays = 30, [int]$TermMonths = 12)

SWRENEW-Report.ps1

Purpose: Produces the Renewal Calendar (date-ordered, with each upcoming renewal placed in a lead-time alert tier: T-120 / T-90 / T-60 / T-30) and a cost rollup (annual cost by vendor and total), from the Contract & Entitlement Repository CSV. OEM bundles are collapsed to one row per BundleId so a bundle is a single renewal decision. Read-only: it never edits the repository. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\SWRENEW-Report.ps1 -CsvPath .\repository.csv -OutDir .\reports EXPECTED RESULTS Two CSVs in -OutDir: renewal-calendar.csv (sorted by RenewalDate, with an AlertTier column) and cost-rollup.csv (annual cost by vendor + grand total). A console summary of due-soon counts per tier and the grand total cost. .NOTES Software Asset Management — repeatable renewal/contract process. Read-only. #> [CmdletBinding()] param([Parameter(Mandatory)][string]$CsvPath, [string]$OutDir = '.\reports')

How to run: powershell -ExecutionPolicy Bypass -File .\SWRENEW-Report.ps1 -CsvPath .\repository.csv -OutDir .\reports EXPECTED RESULTS Two CSVs in -OutDir: renewal-calendar.csv (sorted by RenewalDate, with an AlertTier column) and cost-rollup.csv (annual cost by vendor + grand total). A console summary of due-soon counts per tier and the grand total cost. .NOTES Software Asset Management — repeatable renewal/contract process. Read-only. #> [CmdletBinding()] param([Parameter(Mandatory)][string]$CsvPath, [string]$OutDir = '.\reports')

SWRENEW-Set-ScopeTargeting.ps1

Purpose: Derives the renewal-owner routing from the Contract & Entitlement Repository CSV and writes two routing artifacts used to send lead-time alerts to the right person: - "Renewal Owners - Pilot Scope - Group" (contracts in the pilot product family) - "Renewal Owners - All Products - Group" (every contract with an owner) Each group maps Owner -> the contracts they are responsible for. Optionally creates matching Microsoft Entra security groups via Graph for alert distribution. HOW TO RUN CSV routing only: powershell -ExecutionPolicy Bypass -File .\SWRENEW-Set-ScopeTargeting.ps1 -CsvPath .\repository.csv -PilotProductFamily "Adobe" -OutDir .\reports Also create Entra groups: add -CreateEntraGroups (requires Group.ReadWrite.All) EXPECTED RESULTS Two CSV routing files in -OutDir (pilot and all-products), each listing Owner, contracts, and contract count. With -CreateEntraGroups, the two named security groups are created if missing. .NOTES Software Asset Management — repeatable renewal/contract process. #> [CmdletBinding()] param([Parameter(Mandatory)][string]$CsvPath, [string]$PilotProductFamily, [string]$OutDir = '.\reports', [switch]$CreateEntraGroups)

How to run: CSV routing only: powershell -ExecutionPolicy Bypass -File .\SWRENEW-Set-ScopeTargeting.ps1 -CsvPath .\repository.csv -PilotProductFamily "Adobe" -OutDir .\reports Also create Entra groups: add -CreateEntraGroups (requires Group.ReadWrite.All) EXPECTED RESULTS Two CSV routing files in -OutDir (pilot and all-products), each listing Owner, contracts, and contract count. With -CreateEntraGroups, the two named security groups are created if missing. .NOTES Software Asset Management — repeatable renewal/contract process. #> [CmdletBinding()] param([Parameter(Mandatory)][string]$CsvPath, [string]$PilotProductFamily, [string]$OutDir = '.\reports', [switch]$CreateEntraGroups)

SWRENEW-Validation.ps1

Purpose: Checks every record in the Contract & Entitlement Repository CSV for the required fields (RenewalDate, NoticePeriodDays, AutoRenew, Quantity, AnnualCost) and valid ISO dates, and reports which contracts are incomplete. Read-only: it never edits the repository. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\SWRENEW-Validation.ps1 -CsvPath .\repository.csv EXPECTED

RESULTS "REPOSITORY COMPLETE" (exit 0) when every record has all required fields and valid dates; otherwise a per-contract list of what is missing (exit 1). .NOTES Software Asset Management — repeatable renewal/contract process. Read-only. #> [CmdletBinding()] param([Parameter(Mandatory)][string]$CsvPath)

How to run: powershell -ExecutionPolicy Bypass -File .\SWRENEW-Validation.ps1 -CsvPath .\repository.csv EXPECTED RESULTS "REPOSITORY COMPLETE" (exit 0) when every record has all required fields and valid dates; otherwise a per-contract list of what is missing (exit 1). .NOTES Software Asset Management — repeatable renewal/contract process. Read-only. #> [CmdletBinding()] param([Parameter(Mandatory)][string]$CsvPath)